RESTful Web Services

with Spring REST

Complete Senior Engineer Notes

80% Deep Theory · 20% Code · 80+ Interview Q&A

How It Works · Why It Works · When To Use It

11 Chapters · Deep Internals · Architecture · Production Patterns

Table of Contents

- Pageable Resolution
- Page vs Slice Theory
- OFFSET vs Keyset Pagination

Chapter 10. API Documentation

- OpenAPI 3.0 Theory
- SpringDoc Internals
- Production Documentation Strategy

Chapter 11. REST Best Practices

- URI Design Philosophy
- Versioning Strategies
- Security Checklist
- Idempotency
- Rate Limiting

CHAPTER 1

Introduction to REST & Spring REST

Understanding architectural philosophy before writing a single line of code

1.1 The Birth of REST — A Story

STORY

The year is 2000. Roy Thomas Fielding — one of the principal authors of the HTTP specification itself — is completing his PhD dissertation at UC Irvine. The web has exploded in the 1990s but systems communicating over it are a mess. Everyone has their own protocol: CORBA, DCOM, RMI, SOAP with massive XML envelopes. Fielding asks himself: "What made the World Wide Web so massively scalable? What properties enabled millions of servers and billions of clients to work together without central coordination?" He extracted those properties, named the architecture REST — Representational State Transfer — and published it in Chapter 5 of his dissertation. Today every API you interact with — Swiggy, Google Pay, Instagram, Zomato — follows REST principles.

1.2 What is REST? — The Precise Definition

DEEP DIVE

REST (Representational State Transfer) is an ARCHITECTURAL STYLE — not a protocol, not a library, not a standard. It is a set of constraints. When a system follows all these constraints it is called RESTful. The name itself tells the full story: "Representational" means clients work with REPRESENTATIONS of resources (JSON, XML) not the resources themselves. "State Transfer" means client state is transferred to the server in each request (statelessness). REST is NOT about JSON. REST is NOT about HTTP verbs. Those are implementation details. REST is about architectural constraints that produce scalability, modifiability, visibility, portability and reliability.

1.3 The 6 REST Constraints — Deep Theory

Constraint 1: Statelessness — The Most Critical Constraint

Every request from client to server MUST contain ALL information necessary to understand and process that request. The server MUST NOT store any client session state between requests. Why does this matter architecturally? Imagine Netflix with 200 million users. If the server stored session state for every user, every request would need to reach the SAME server (sticky sessions). You cannot add more servers freely — you are locked in. With statelessness, ANY server in a cluster of 1000 can handle ANY request from ANY user because the request carries everything needed. This is why AWS, Google and Netflix can horizontally scale to thousands of servers effortlessly. Statelessness IS scalability.

INTERN

INTERN INSIGHT: Stateless does NOT mean the server has no database. It means no per-CLIENT session state is stored in server memory between requests. The database stores persistent data — that is fine. What is NOT fine: server keeping a Map of sessionId to UserState in memory. Solution in practice: JWT tokens — the client holds the token, sends it every request. Server verifies the token without needing to remember anything about the client.

Constraint 2: Client-Server Separation

UI concerns must be separated from data storage concerns. Client and server evolve independently. iOS app, Android app, React SPA and CLI tool can all use the SAME server API simultaneously. The server team and frontend team can deploy independently. This separation is why Swiggy can redesign its mobile app without touching the backend — and vice versa.

Constraint 3: Cacheability

Responses must define themselves as cacheable or non-cacheable. If cacheable, the client (or intermediate proxies like CDNs) can reuse responses for equivalent future requests. This eliminates server round-trips, reduces latency and improves scalability. HTTP provides Cache-Control, ETag, Last-Modified, and Expires headers for this. When Swiggy restaurant list loads instantly on your second visit — that is caching working.

Constraint 4: Uniform Interface — The Central REST Feature

This is what makes REST distinctive. It has four sub-constraints that must ALL be satisfied:

- (a) Resource Identification: Individual resources are identified in requests using URIs. The resource itself (a database row) is conceptually different from its representation (JSON). GET /users/42 identifies a resource; the JSON returned is just a representation of it.
- (b) Manipulation Through Representations: When a client holds a representation of a resource including metadata, it has enough information to modify or delete the resource on the server.
- (c) Self-Descriptive Messages: Each message includes enough information to describe how to process it. Content-Type header tells the receiver how to parse the body. HTTP status codes describe outcome semantics.
- (d) HATEOAS — Hypermedia as the Engine of Application State: Responses include hyperlinks to related actions. A client starts at a known URL and discovers everything else through links — just like a human navigates a website without knowing every URL in advance.

Constraint 5: Layered System

The client cannot tell whether it is connected directly to the origin server or to an intermediary like a load balancer, CDN, API gateway or caching proxy. Your mobile app calling api.swiggy.com might hit AWS CloudFront (CDN), then an API Gateway, then a load balancer, then one of 500 application servers — and the app is completely unaware. This allows operators to add infrastructure layers transparently.

Constraint 6: Code on Demand (Optional)

Servers can extend client functionality by sending executable code such as JavaScript. This is the only optional REST constraint. Most REST APIs do not use it — a webpage loading JavaScript from a server is the classic example.

1.4 How Spring Internally Implements REST — The Complete Chain

DEEP DIVE

THE REQUEST LIFECYCLE EXPLAINED STEP BY STEP: 1. HTTP Request arrives at the JVM process running embedded Tomcat. 2. Tomcat NIO Connector accepts the TCP connection and parses raw HTTP bytes into a ServletRequest object. Tomcat uses a thread pool (default 200 threads) — each request gets one thread. 3. DispatcherServlet intercepts ALL requests (mapped to "/" in Spring Boot). This is the Front Controller pattern — single entry point. Spring Boot auto-registers it via DispatcherServletAutoConfiguration. 4. RequestMappingHandlerMapping is consulted. It scanned all @RestController classes at startup and built a URL-to-method lookup table. It finds your @GetMapping method in O(1). 5. RequestMappingHandlerAdapter invokes the controller method. Before calling, it resolves each parameter using HandlerMethodArgumentResolvers. 6. @RequestBody processing: RequestResponseBodyMethodProcessor reads the request body stream and delegates to MappingJackson2HttpMessageConverter which uses ObjectMapper to deserialize JSON bytes into your Java DTO via reflection. 7. Your controller method body executes. Service, repository, database. 8. Return value goes back through RequestResponseBodyMethodProcessor which serializes it to JSON using the same ObjectMapper and writes bytes to the HTTP response. 9. Tomcat sends the HTTP Response back to the client.

1.5 HTTP Methods and REST Semantics

HTTP Method	REST Semantic	Safe?	Idempotent?
GET	Retrieve resource(s)	Yes — no side effects	Yes — same result every call
POST	Create new resource	No	No — duplicates on retry
PUT	Full replace of resource	No	Yes — replacing with same data = same result
PATCH	Partial update	No	Depends on implementation
DELETE	Remove resource	No	Yes — deleting already-deleted = 404 or 204
HEAD	Same as GET, no body	Yes	Yes
OPTIONS	What methods are allowed	Yes	Yes

Interview Questions — Chapter 1

Q1. What is REST and how does it differ from SOAP?

Ans: REST is an architectural style using HTTP with lightweight JSON/XML. It is stateless, cacheable and leverages existing HTTP semantics. SOAP is a formal protocol with rigid XML envelope structure, WSDL contracts, and WS-* standards for security and reliability. REST is simpler and better for internet-facing APIs. SOAP is still used in enterprise banking and payment systems where formal contracts, ACID transactions and built-in error handling (SOAP Faults) are required. The key mindset difference: SOAP exposes operations (verbs); REST exposes resources (nouns).

Q2. Explain the Uniform Interface constraint and why it is the central feature of REST.

Ans: Uniform Interface simplifies and decouples the architecture by imposing four constraints: (1) Resource identification via URIs — every entity has a stable address. (2) Manipulation through representations — clients use the representation to modify the resource, not direct database access. (3) Self-descriptive messages — each message carries processing instructions in headers (Content-Type, Cache-Control). (4) HATEOAS — responses contain links to next possible actions, making the API self-discoverable. The power: any HTTP-capable client on earth can interact with any REST server without prior knowledge of its internal implementation.

Q3. What does statelessness mean? Can a REST API use a database?

Ans: Statelessness means the SERVER stores no client session state between requests. Every request is self-contained. Yes, the database stores persistent data — that is completely fine. What is forbidden: server keeping user shopping cart, authentication state, or wizard step in server memory between requests. In practice: JWT tokens carry identity in every request (client stores the token). This distinction — statelessness vs. no persistence — confuses many juniors.

Q4. What is HATEOAS and is it commonly implemented?

Ans: Hypermedia As The Engine Of Application State: responses include links to related actions, making the API self-discoverable like a website. In theory, clients can navigate the API without pre-baked URL knowledge. In practice, full HATEOAS adds significant complexity and most enterprise teams correctly stop at Level 2 (HTTP verbs + proper status codes). It IS Level 3 of the Richardson Maturity Model and knowing it well impresses interviewers.

Q5. What is the difference between safe and idempotent HTTP methods?

Ans: Safe means the method does not change server state — GET, HEAD, OPTIONS are safe. Idempotent means calling it N times has the same effect as calling once. GET, HEAD, PUT, DELETE are idempotent. POST is NEITHER safe nor idempotent. Why it matters: HTTP clients can automatically retry safe and idempotent requests on network failure. Retrying a POST without idempotency keys creates duplicate orders — a serious production bug.

CHAPTER 2

Setting Up Spring REST

From zero to running API in 2 minutes — understanding the auto-configuration magic

2.1 The Problem Spring Boot Solved — Historical Context

STORY

Before Spring Boot (before 2014), building a Spring MVC REST API required: downloading Apache Tomcat separately and installing it, writing a web.xml deployment descriptor, writing a Spring application context XML file (100-200 lines), configuring DispatcherServlet in XML, manually adding 15-20 Maven dependencies, and deploying a WAR file to the external Tomcat. This process took 2-4 hours for an experienced developer. Spring Boot (created by Phil Webb and Dave Syer at Pivotal) inverted this: one dependency, zero XML, embedded server, executable JAR. Zero-to-running in under 2 minutes. That single dependency is spring-boot-starter-web.

2.2 What spring-boot-starter-web Actually Does — Internal Theory

A "starter" in Spring Boot is a specially crafted Maven/Gradle artifact that is primarily a dependency aggregator — a POM with only dependencies and no code of its own. spring-boot-starter-web transitively brings approximately 67 jars into your project. The important ones:

Included Library	Purpose in REST APIs
spring-webmvc	Core MVC framework: DispatcherServlet, HandlerMapping, ViewResolver, MessageConverters
spring-boot-starter-tomcat	Embedded Tomcat 10.x — starts inside your JVM process, no external server needed
jackson-databind	JSON serialization/deserialization — converts Java objects to/from JSON
jackson-datatype-jsr310	Java 8+ date/time (LocalDate, Instant) support in Jackson serializer
spring-boot-autoconfigure	Auto-configuration: reads classpath at startup and auto-creates 100+ beans conditionally
spring-boot-starter-logging	Logback as default logging framework, SLF4J as the facade
spring-core, spring-context	IoC container, ApplicationContext, dependency injection

2.3 Auto-Configuration — The Mechanism Explained

DEEP DIVE	AUTO-CONFIGURATION MECHANISM STEP BY STEP: 1. @SpringBootApplication on your main class includes @EnableAutoConfiguration. 2. At startup, Spring reads META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports from every jar on the classpath. This file lists 100+ configuration classes. 3. Each auto-configuration class is annotated with @Conditional annotations: @ConditionalOnClass (only activate if certain class is on classpath), @ConditionalOnMissingBean (only create bean if you have not defined your own), @ConditionalOnProperty (only if a property is set). 4. DispatcherServletAutoConfiguration fires because DispatcherServlet.class is on the classpath (from spring-webmvc) and you have not defined your own DispatcherServlet bean. 5. JacksonAutoConfiguration fires because ObjectMapper.class is on the classpath (from jackson-databind) and no custom ObjectMapper bean exists. 6. EmbeddedWebServerFactoryCustomizerAutoConfiguration fires and configures TomcatServletWebServerFactory to start Tomcat on port 8080. KEY INSIGHT: You can override ANY auto-configured bean by defining your own @Bean. This is "Convention over Configuration" — sensible defaults, easy override.

2.4 Embedded Tomcat — Architecture and Implications

Traditional Java web apps deployed as WAR files to an external Tomcat server. Spring Boot flips this: Tomcat is embedded INSIDE your application as a library. When you run the JAR, your Java main() method programmatically creates and starts Tomcat. TomcatServletWebServerFactory creates a Tomcat instance, configures the NIO connector on port 8080, registers your DispatcherServlet, and calls tomcat.start(). The entire web server lifecycle is managed by Spring.

Implication for production: you package a single executable FAT JAR containing your code + all dependencies + embedded Tomcat (typically 15-20MB total). Deployment becomes simply: java -jar myapp.jar. This made Docker containers practical — one JAR, one container image, infinitely deployable. You can switch to Jetty or Undertow by excluding spring-boot-starter-tomcat and adding the alternative — zero code changes.

2.5 application.properties — Key Configuration Explained

Spring Boot uses a property-based configuration system with a strict hierarchy. Properties can come from (in priority order, highest first): command-line arguments, OS environment variables, application-{profile}.properties, application.properties, and Spring defaults. This hierarchy enables the Twelve-Factor App methodology: same JAR, different config per environment.

```
# Server configuration
server.port=8080
server.servlet.context-path=/api # All URLs prefixed with /api
server.tomcat.max-threads=200 # Thread pool size
# Jackson — JSON serialization behavior
spring.jackson.default-property-inclusion=non_null # Omit null fields
spring.jackson.serialization.write-dates-as-timestamps=false # ISO 8601 dates
spring.jackson.deserialization.fail-on-unknown-properties=false # Ignore extra fields
# JPA / Database
spring.jpa.show-sql=true # Dev only — prints SQL to console
spring.jpa.hibernate.ddl-auto=validate # PRODUCTION: validates schema, never drops
```

```
# Error handling — never expose stack traces to clients
server.error.include-stacktrace=never
server.error.include-exception=false
```

WARNING CRITICAL: Never use spring.jpa.hibernate.ddl-auto=create or create-drop in production. These DROP all tables at startup — you will lose all production data. Use validate in production and Flyway/Liquibase for schema migrations.

Interview Questions — Chapter 2

Q1. How does Spring Boot auto-configuration work internally?

Ans: Spring Boot scans META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports in every classpath jar. Each entry is a @Configuration class annotated with @Conditional annotations. DispatcherServletAutoConfiguration creates DispatcherServlet only if (a) the spring-webmvc jar is present AND (b) you have not already defined a DispatcherServlet bean. This is @ConditionalOnMissingBean in action. You override any auto-configured bean by defining your own — Spring backs off automatically.

Q2. What is the difference between application.properties and application.yml?

Ans: Both externalize configuration. yml uses YAML hierarchical syntax (more readable for nested properties, less repetition). properties uses flat key=value pairs. Spring Boot supports both. Both support profile-specific variants: application-dev.yml, application-prod.properties. yml is preferred in cloud-native and Kubernetes environments because it maps naturally to K8s ConfigMaps. Pick one style and stay consistent throughout the project.

Q3. How do you switch from embedded Tomcat to Jetty?

Ans: Exclude spring-boot-starter-tomcat from spring-boot-starter-web in pom.xml and add spring-boot-starter-jetty. Zero code changes — auto-configuration detects Jetty on the classpath and creates JettyServletWebServerFactory instead of TomcatServletWebServerFactory. This works because Spring Boot uses @ConditionalOnClass to pick the right server factory. Undertow is another option — better throughput for non-blocking/reactive workloads.

Q4. What does spring.jpa.hibernate.ddl-auto=validate do and why is it right for production?

Ans: validate: Hibernate reads your @Entity class mappings and compares them against the actual database schema at startup. If they do not match (missing column, wrong type), the app FAILS TO START immediately — catching schema drift before any traffic hits. This is the correct production setting. In dev use update (auto-migrate) or create-drop (fresh DB each restart). Never use create or drop in production — those destroy your data. Use Flyway or Liquibase for production schema migrations.

CHAPTER 3

Richardson Maturity Model

The 4 levels of REST enlightenment — measuring how RESTful your API really is

3.1 The Story — Not All REST APIs Are Equal

STORY

In 2008 Leonard Richardson analyzed hundreds of real-world APIs and noticed something disturbing: developers were claiming their APIs were "RESTful" while violating most REST constraints. A classic example: POST /api with body {"action":"getUser","id":42}. That is using HTTP as a transport pipe for RPC calls — not REST at all. Richardson created his Maturity Model to provide a measurable progression from RPC-over-HTTP to true hypermedia systems. Think of it as martial arts belts: white to black. Most APIs in production today are Level 2. True Level 3 is extremely rare but architecturally significant.

3.2 Level 0 — The Swamp of POX

POX stands for Plain Old XML (or JSON). This is pre-REST territory. One URL handles everything. HTTP is just a tunnel used as a transport protocol, not leveraged for its semantic richness. All requests are typically POST. The action to perform is encoded in the request body. This is essentially RPC (Remote Procedure Call) over HTTP — the same pattern as old SOAP services. There are no resources, no HTTP verbs used meaningfully, no status code semantics. The server just reads the body and figures out what to do.

3.3 Level 1 — Resources

Progress! Each resource now has its own URI. Instead of one /api endpoint handling everything, you have /users, /orders, /products. But the HTTP verb is still not used meaningfully — POST is used for everything: creating, reading, updating, deleting. You have identified your resources but not yet leveraged the HTTP protocol's rich verb semantics. This is common in older SOAP-to-REST migrations.

3.4 Level 2 — HTTP Verbs + Status Codes (The Industry Standard)

This is where 95% of production APIs live and it is the correct target for most teams. Level 2 uses HTTP verbs with their full semantic meaning: GET for retrieval (safe, idempotent, cacheable), POST for creation (not idempotent), PUT for full replacement (idempotent), PATCH for partial update, DELETE for removal (idempotent). Additionally, HTTP status codes are used precisely — 200 for success, 201 for creation, 204 for no-content operations, 400 for bad requests, 404 for not found, 500 for server errors. This level fully leverages the HTTP protocol as an application layer, not just a transport.

TIP

WHY LEVEL 2 IS ENOUGH: HTTP verbs and status codes give clients rich semantic information. A caching proxy knows GET requests are safe to cache. A client knows 201 means a new resource was created and checks the Location header. An API gateway knows 500 means retry with backoff. This shared understanding is why Level 2 is the production sweet spot.

3.5 Level 3 — Hypermedia Controls (True REST)

This is Fielding's "true REST." The response includes not just data but hyperlinks to related actions and resources — exactly like HTML web pages link to other pages. A client needs to know only ONE entry point URL. Everything else is discoverable through links in the responses. This makes the API self-documenting and allows server-side URL restructuring without breaking clients since clients follow links rather than using hardcoded URLs. The tradeoff: massive implementation complexity, verbose responses, and clients must be built to follow links dynamically.

Level	Name	Description	Real Example
0	Swamp of POX	One URL, HTTP as tunnel, all POST	POST /api {"action":"getUser","id":42}
1	Resources	Individual URIs per resource, still POST for everything	POST /users/42 to read user 42
2	HTTP Verbs	Correct HTTP verbs + proper status codes. 95% of APIs.	GET /users/42 → 200 OK, DELETE /users/42 → 204
3	HATEOAS	Response includes links to next possible actions	{"id":42,"_links":{"self":"/users/42","orders":"/users/42/orders"}}

Interview Questions — Chapter 3

Q1. What level of Richardson Maturity should a production API target and why?

Ans: Level 2 is the correct target. It uses HTTP verbs with proper semantics, meaningful status codes and clean URI design. This gives clients rich semantic information, enables caching, and leverages the entire HTTP ecosystem (proxies, gateways, browsers). Level 3 HATEOAS adds discoverability but at the cost of verbose responses, complex client code (must parse and follow links rather than using known URLs) and significant implementation effort. A senior engineer who says "we target Level 2 and would consider HATEOAS for scenarios where client URL discoverability genuinely matters" gives the right answer.

Q2. What is the difference between PUT and PATCH?

Ans: PUT is idempotent full replacement — you send the ENTIRE resource representation. If you omit a field, that field is set to null/default on the server. PATCH is partial update — you send ONLY the fields you want to change. Best practice: use PATCH for partial updates to avoid accidentally clearing fields the client did not intend to change. Example: PATCH /users/42 with {"email":"new@x.com"} changes only email. PUT /users/42 without the email field would clear it.

Q3. How would you explain HATEOAS to a non-technical person?

Ans: Think of it like a restaurant menu with QR codes. You scan one QR code (the entry URL) and the menu shows every option available, with buttons to place an order, call a waiter, or see the bill. You never need to memorize any URLs — everything is linked. Without HATEOAS it is like the waiter brings you food and tells you nothing about what else you can order. The API can change all its internal URLs without breaking your app because your app follows links rather than hardcoding addresses.

CHAPTER 4

Spring REST Controllers

@RestController internals, URL mapping, argument resolution — the full anatomy

4.1 What @RestController Really Is — Under the Hood

@RestController is a composed annotation — a meta-annotation that combines @Controller and @ResponseBody into one. @Controller tells Spring: this class is a web controller, register it in the application context and make it eligible for request handling. @ResponseBody tells Spring: for every method in this class, instead of treating the return value as a view name and looking up a template (Thymeleaf, JSP), write the return value directly to the HTTP response body. Without @ResponseBody, returning a String "users/list" from a controller method would tell Spring to find a view file called users/list.html.

Internally, when Spring's component scanning finds @RestController, it registers the class with RequestMappingHandlerMapping. That mapping scans all methods annotated with @RequestMapping (or its composed variants) and builds an internal lookup structure mapping HTTP method + URL pattern to a class and method reference. This lookup is built ONCE at startup and used for every request during the application lifetime.

4.2 @RequestMapping and Its Composed Variants

@GetMapping, @PostMapping, @PutMapping, @PatchMapping and @DeleteMapping are all composed annotations on top of @RequestMapping with the method attribute pre-set. They are pure syntactic sugar introduced in Spring 4.3 for readability and intent clarity. At runtime Spring processes them identically — they all resolve to @RequestMapping with a specific HTTP method constraint. Class-level @RequestMapping sets a base path. Method-level annotations append to it. If the class has @RequestMapping("/api/v1/users") and a method has @GetMapping("/{id}"), the effective URL is /api/v1/users/{id}. Changing the class-level path changes ALL endpoints in that controller at once.

4.3 How Spring Resolves Method Arguments — The Full Theory

DEEP DIVE This is one of the most important Spring internals to understand. When RequestMappingHandlerAdapter is about to invoke your controller method, it must supply values for each parameter. It iterates through a registered list of HandlerMethodArgumentResolver implementations, asking each one: "Can you resolve this parameter?" The first one that says yes handles it. This is the Strategy design pattern in action.

Annotation	Resolver Class	How It Works
@PathVariable	PathVariableMethodArgumentResolver	Extracts URI template variables parsed from URL by AntPathMatcher
@RequestParam	RequestParamMethodArgumentResolver	Reads from HttpServletRequest.getParameter() — query string or form data

@RequestBody	RequestResponseBodyMethodProcessor	Reads request InputStream, selects HttpMessageConverter by Content-Type, uses Jackson to deserialize
@RequestHeader	RequestHeaderMethodArgumentResolver	Reads from HttpServletRequest.getHeader()
Pageable	PageableHandlerMethodArgumentResolver	Reads page, size, sort params and constructs PageRequest object
HttpServletRequest	ServletRequestMethodArgumentResolver	Injects the raw Servlet request object directly

4.4 @PathVariable vs @RequestParam — Design Philosophy

This is not just a technical choice — it is a REST design philosophy. Path variables (/users/42) make the resource identity part of the URL itself. The URL IS the address of a specific resource. Query parameters (/users?status=active) are filters, modifiers or optional instructions applied to a collection. Use path variables when the parameter IDENTIFIES the resource. Use query parameters when the parameter FILTERS or MODIFIES the collection.

Annotation	Use When	Example URL	Example Code
@PathVariable	Identifying a SPECIFIC resource — it is part of the resource identity	/users/42	"/{id}" + @PathVariable Long id
@RequestParam	Filtering, searching, sorting, optional params with defaults	/users?name=Rahul&page;=0	"?name=Rahul" + @RequestParam String name

```
// Complete controller showing every annotation type:
@RestController
@RequestMapping("/api/v1/users")
public class UserController {
@GetMapping("/{id}") // PathVariable — identifies specific resource
public ResponseEntity getUser(@PathVariable Long id) { ... }
@GetMapping // RequestParam — filters the collection
public Page list(@RequestParam(defaultValue="") String name,
Pageable pageable) { ... }
@PostMapping // RequestBody — deserializes JSON to Java DTO
@ResponseStatus(HttpStatus.CREATED)
public UserDto create(@Valid @RequestBody UserCreateDto dto) { ... }
}
```

Interview Questions — Chapter 4

Q1. What is the difference between @Controller and @RestController?

Ans: @Controller is for controllers that render views (HTML pages). It returns view names resolved by a ViewResolver to find templates (Thymeleaf, JSP). @RestController = @Controller + @ResponseBody. @ResponseBody instructs Spring: skip view resolution and write the return value directly to the HTTP response body, serialized by an HttpMessageConverter (Jackson to JSON). Use @RestController for all REST APIs. Use @Controller for server-side rendered web applications.

Q2. How does Spring match incoming URLs to controller methods?

Ans: At startup, RequestMappingHandlerMapping inspects all @RestController beans, reads every @RequestMapping annotation and builds a MappingRegistry — a map from HTTP method + URL pattern to HandlerMethod. URL patterns support Ant-style wildcards. When a request arrives, DispatcherServlet asks the mapping to find the best match using a scored algorithm — more specific patterns win. No match gives 404. Wrong HTTP method gives 405 Method Not Allowed.

Q3. What happens when @RequestBody JSON has extra unknown fields?

Ans: By default Jackson throws UnrecognizedPropertyException which becomes HTTP 400. Fix globally: set spring.jackson.deserialization.fail-on-unknown-properties=false in properties. Or add @JsonIgnoreProperties(ignoreUnknown=true) on the DTO class. Best practice for production: ignore unknown properties globally. This enables forward compatibility — clients sending newer fields to an older API version do not get 400 errors.

Q4. Can two controller methods have the same URL?

Ans: No — it causes BeanCreationException at startup: "Ambiguous handler methods mapped." Spring fails fast during context initialization. You CAN have the same URL with different HTTP methods: GET /users and POST /users are distinct mappings because both URL AND HTTP method are part of the mapping key. Same URL, same method = always an error.

CHAPTER 5

Request & Response Handling

Jackson internals, ResponseEntity patterns, HTTP status code theory, content negotiation

5.1 Jackson — The JSON Engine Explained Deeply

Jackson (by FasterXML) is embedded in Spring Boot as the default JSON library. It is not just a JSON printer — it is a full serialization framework supporting JSON, XML, YAML and CSV through different modules. The core class is ObjectMapper, which Spring Boot creates as a singleton bean via JacksonAutoConfiguration. This singleton is shared across all HTTP message conversion — every request and response goes through this one ObjectMapper instance.

DEEP DIVE

HOW JACKSON SERIALIZES A JAVA OBJECT TO JSON (Step by Step Internals): 1. ObjectMapper.writeValueAsBytes(yourObject) is called. 2. Jackson inspects the class using reflection to discover fields and properties. 3. For each field: if public field — use directly. If private with getter — use the getter method. If annotated with @JsonProperty — use the specified JSON key name. 4. Jackson checks @JsonIgnore — skip this field completely. @JsonInclude(NON_NULL) — skip if value is null. 5. For special types: LocalDate/LocalDateTime use the JavaTimeModule serializer if registered. Without JavaTimeModule, LocalDate is serialized as a JSON array [2024,1,15] — a very common bug in new Spring Boot apps. 6. The JSON bytes are written to the output stream of the HTTP response. DESERIALIZATION (JSON to Java) is the reverse: JSON bytes go through JsonParser, token by token, into Java object construction using the default no-arg constructor, then setter calls for each field, or @JsonCreator for immutable objects.

5.2 ResponseEntity — The Correct Way to Control HTTP Responses

ResponseEntity<T> is a generic class representing the complete HTTP response: status code, headers, and body together. It gives you fine-grained control that raw return types do not provide. When you return a plain User object from a controller, Spring returns 200 OK with the serialized body — but you cannot set headers, add location, or return 404 conditionally without throwing exceptions. ResponseEntity solves this elegantly and expressively.

```
// Pattern 1 — Conditional 200/404 (most common real-world usage)
return userRepo.findById(id)
.map(ResponseEntity::ok)
.orElse(ResponseEntity.notFound().build());
// Pattern 2 — 201 Created with Location header (REST standard for POST)
URI location = ServletUriComponentsBuilder.fromCurrentRequest()
.path("/{id}").buildAndExpand(saved.getId()).toUri();
return ResponseEntity.created(location).body(saved);
// Pattern 3 — Custom headers for pagination metadata
return ResponseEntity.ok()
.header("X-Total-Count", String.valueOf(total))
.header("X-Page-Number", String.valueOf(page))
.body(pageData);
```

5.3 HTTP Status Codes — The Semantic Contract

HTTP status codes are not mere numbers — they are a semantic contract between client and server. They tell every component in the chain (client, proxy, CDN, API gateway, monitoring system, circuit breaker) what happened and what to do next. Using wrong status codes breaks this contract and creates subtle bugs that are very hard to diagnose.

Code	Name	When to Return — Precisely
200	OK	GET success with body, PUT/PATCH success when returning body
201	Created	POST creates new resource. MUST include Location header pointing to new resource.
204	No Content	DELETE success, PUT/PATCH when not returning body. No body allowed.
400	Bad Request	Invalid request syntax, @Valid validation failure, malformed JSON.
401	Unauthorized	Not authenticated — missing/expired/invalid JWT or credentials.
403	Forbidden	Authenticated but not authorized. @PreAuthorize role check failed.
404	Not Found	Specific resource ID does not exist. NOT for empty collections.
405	Method Not Allowed	POST to GET-only endpoint. Spring returns this automatically.
409	Conflict	Duplicate email, concurrent modification, optimistic lock failure.
422	Unprocessable	JSON is syntactically valid but business rule violated.
429	Too Many Requests	Rate limit exceeded. Include Retry-After header.
500	Internal Server Error	Unhandled exception. NEVER expose stack trace. Log server-side.
503	Service Unavailable	Database down, circuit breaker open. Include Retry-After header.

5.4 Content Negotiation — Theory

Content negotiation is how client and server agree on the format of the response. The client sends an Accept header (Accept: application/xml) expressing its preference. Spring's ContentNegotiatingViewResolver checks the list of registered HttpMessageConverters and finds one that can produce the requested media type. If no converter can satisfy the Accept header, Spring returns 406 Not Acceptable. Add jackson-dataformat-xml dependency to support both JSON and XML with zero code changes — auto-configuration picks it up.

Interview Questions — Chapter 5

Q1. What is the difference between 401 Unauthorized and 403 Forbidden?

Ans: 401 means the request lacks valid authentication credentials — the server does not know who you are. The client should provide credentials (or refresh the JWT) and retry. The name "Unauthorized" is misleading — it really means "unauthenticated." 403 means the server knows exactly who you are (you are authenticated) but you do not have permission to access this specific resource. The client should NOT retry — no amount of token refreshing will grant access you do not have. Spring Security returns 401 for invalid/missing JWT via AuthenticationEntryPoint and 403 for insufficient roles via AccessDeniedHandler.

Q2. Why should POST return 201 with a Location header instead of 200?

Ans: 200 OK means "I did the thing and here is the result." 201 Created carries additional semantics: a new resource was created, and the Location header tells the client exactly where to find it. This follows RFC 7231. Benefits: (1) Clients can immediately GET the resource without parsing the body. (2) API gateways can count creation events specifically. (3) Caching proxies handle it correctly as a new resource. Always include Location: /api/v1/users/42 on 201 responses.

Q3. How does Jackson handle LocalDate serialization and what is the common bug?

Ans: Without JavaTimeModule, Jackson serializes LocalDate as a JSON array [2024, 1, 15] — completely unusable by most clients. Solution: spring-boot-starter-web auto-registers JavaTimeModule via JacksonAutoConfiguration. Set spring.jackson.serialization.write-dates-as-timestamps=false to get ISO 8601 strings (2024-01-15) instead of arrays. This is one of the most common Jackson bugs in Spring Boot apps because it works fine in unit tests (where you might not test the actual JSON output) but fails when consumed by real clients.

Q4. What is the difference between @ResponseBody and ResponseEntity?

Ans: @ResponseBody on a method means Spring serializes the return value to the response body with default status 200 and no custom headers. ResponseEntity<T> gives you explicit control over status code, headers AND body in a single return statement. Use ResponseEntity when you need: conditional 404 vs 200, 201 Created with Location header, 204 No Content, custom response headers, or different status codes based on business logic. ResponseEntity is always preferred for REST APIs — it makes HTTP semantics explicit.

CHAPTER 6

Error & Exception Handling

From scattered try-catch to centralized, secure, consistent error responses

6.1 The Problem with Unstructured Exception Handling

STORY

STORY: A startup built a Spring REST API with 40 controllers. Each developer handled exceptions their own way. Controller A caught ResourceNotFoundException and returned 404. Controller B let the same exception propagate — the client got Spring's HTML white-label error page with a Java stack trace embedded in it. That stack trace exposed: Spring version (known CVEs), class names (internal architecture), and exact error message (SQL query details). A security researcher found it in 3 hours and reported it as a medium-severity vulnerability. The fix: adding @RestControllerAdvice — 50 lines replaced 400 lines of scattered try-catch and eliminated the security risk.

6.2 @RestControllerAdvice — Architecture and Mechanism

@RestControllerAdvice is a specialization of @ControllerAdvice (which is itself a specialization of @Component). It marks a class as a global exception handler that applies to ALL controllers in the application context. The @ResponseBody meta-annotation means all return values from its methods are serialized to JSON — not treated as view names. This gives you one place for all error handling across the entire application.

DEEP DIVE

HOW SPRING ROUTES EXCEPTIONS TO HANDLER METHODS: When an exception propagates out of a controller method (either thrown directly or bubbled up from service/repository), Spring's ExceptionHandlerExceptionResolver intercepts it. It looks through all @ControllerAdvice classes for an @ExceptionHandler method whose declared exception type matches the thrown exception (or its supertype — most specific match wins). If found, it invokes that handler method. If no handler is found, Spring falls back to DefaultHandlerExceptionResolver, then finally to BasicErrorController which produces the dreaded white-label error page. EXCEPTION FLOW STEP BY STEP: 1. Client sends POST /users/99/orders 2. UserService.findById(99) calls repo.findById(99) which returns Optional.empty() 3. Service throws new ResourceNotFoundException("User", 99) 4. RuntimeException propagates: UserService → OrderController → HandlerAdapter → DispatcherServlet 5. ExceptionHandlerExceptionResolver scans @RestControllerAdvice classes 6. Finds handleNotFound(ResourceNotFoundException) method — exact type match 7. Invokes the handler which builds ErrorResponse with status=404, message, timestamp, path 8. @ResponseStatus(NOT_FOUND) sets the response status to 404 9. Jackson serializes ErrorResponse to JSON 10. Client receives: HTTP 404 + {"status":404,"error":"Not Found","message":"User not found with id: 99","timestamp":"..."}

6.3 Standard Error Response Structure

Consistency in error responses is critical for API consumers. Define a standard ErrorResponse DTO and use it across ALL error scenarios. Include: HTTP status code (integer), error category string, human-readable message,

request path that caused the error, timestamp. For validation errors: a map of field name to error message so the client knows exactly which field failed.

```
// Standard ErrorResponse DTO
public class ErrorResponse {
private int status; // 404, 400, 500
private String error; // "Not Found", "Bad Request"
private String message; // Human-readable explanation
private String path; // /api/v1/users/99
private LocalDateTime timestamp;
private Map fieldErrors; // for @Valid failures
}
@RestControllerAdvice
public class GlobalExceptionHandler {
@ExceptionHandler(ResourceNotFoundException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public ErrorResponse handleNotFound(ResourceNotFoundException ex, HttpServletRequest req) {
return new ErrorResponse(404,"Not Found",ex.getMessage(),req.getRequestURI(),now());
}
@ExceptionHandler(Exception.class) // catch-all safety net
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ErrorResponse handleAll(Exception ex) {
log.error("Unhandled exception", ex); // full trace in server logs only
return new ErrorResponse(500,"Internal Server Error","Unexpected error",null,now());
}
}
```

WARNING SECURITY: NEVER send the Java stack trace to the client. It reveals framework versions (known CVEs), internal class names (architecture leak) and exact error messages (may contain SQL). Log the full exception server-side with log.error("", ex). Send only a generic message to the client.

Interview Questions — Chapter 6

Q1. What is the exception handling precedence — what gets caught first?

Ans: Spring evaluates @ExceptionHandler methods from most-specific to least-specific exception type. If you have handlers for both Exception and RuntimeException, and a NullPointerException is thrown, Spring picks RuntimeException handler first (more specific). Spring's own DefaultHandlerExceptionResolver handles standard Spring exceptions (like MethodArgumentNotValidException) BEFORE your custom advice unless you override them. Multiple @ControllerAdvice classes: use @Order annotation to control priority.

Q2. What exception is thrown when @Valid fails and how do you extract field errors?

Ans: MethodArgumentNotValidException is thrown. It contains a BindingResult. To extract errors: ex.getBindingResult().getFieldErrors() returns List. Each FieldError has getField() (field name like "email") and getDefaultMessage() (validator message like "must be a valid email"). Collect into Map and include in error response. This tells the client WHICH field failed and WHY — essential for form validation feedback.

Q3. @ExceptionHandler in the controller class vs in @RestControllerAdvice — what is the difference?

Ans: @ExceptionHandler inside a @RestController class handles exceptions ONLY from that specific controller. @ExceptionHandler inside @RestControllerAdvice handles exceptions from ALL controllers application-wide. Rule: NEVER put @ExceptionHandler in individual controllers — it creates inconsistent error handling. ALL exception handlers belong in a single @RestControllerAdvice class. The controller-level @ExceptionHandler has no practical use case in production REST APIs.

CHAPTER 7

REST Validation

Bean Validation 3.0, Hibernate Validator internals, custom constraints — stopping bad data at the gate

7.1 The Architecture of Validation in Spring

Spring's validation integrates the Java Bean Validation specification (JSR-380, also called Bean Validation 3.0 for Jakarta EE). Hibernate Validator is the reference implementation. Adding spring-boot-starter-validation pulls in hibernate-validator and jakarta.validation-api. Spring Boot auto-configures LocalValidatorFactoryBean — a bridge between Spring's validation system and the Hibernate Validator engine.

DEEP DIVE

VALIDATION TRIGGER CHAIN (what happens when @Valid is used on @RequestBody): 1. DispatcherServlet routes request to controller method. 2. RequestResponseBodyMethodProcessor reads and deserializes JSON to your DTO. 3. SmartValidator (backed by Hibernate Validator) is invoked with the DTO object. 4. Hibernate Validator uses reflection to inspect each field annotated with constraint annotations. 5. For each annotation it looks up the ConstraintValidator implementation for that annotation. Example: @Email maps to EmailValidator, @NotBlank maps to NotBlankValidator. 6. Each validator's isValid() method is called. If false → a ConstraintViolation is added. 7. If ANY constraints fail → MethodArgumentNotValidException is thrown with ALL violations (not just first — Hibernate Validator is fail-slow by default, collecting all errors). 8. Your @ExceptionHandler catches this and builds the field → error message map. CRITICAL: Your controller method body NEVER executes if validation fails. Bad data is stopped at the gate.

7.2 Constraint Annotations — Theory and Semantics

Annotation	Passes When	Common Usage
@NotNull	Value is not null	Numeric, boolean, date, object fields
@NotEmpty	Not null AND size > 0	String, Collection, Array, Map
@NotBlank	Not null AND trimmed length > 0 (strictest)	Text fields: name, description, email
@Size(min,max)	Length/size between min and max	String character count, Collection element count
@Min(n) / @Max(n)	Numeric value >= n or <= n	Age, quantity, price bounds
@Email	Matches email regex format	Email fields
@Pattern(regexp)	Matches the provided Java regex	Phone numbers, postal codes, custom formats
@Past / @Future	Date is in past / future	Booking dates, expiry dates
@Positive / @Negative	Value > 0 / < 0	Amounts, quantities that cannot be zero or negative

@Valid	Cascades validation into nested object	Nested DTO fields — required for nested validation

7.3 Custom Constraint — Theory and Implementation

When built-in annotations do not cover your business rule, create a custom constraint. A custom constraint has two parts: (1) a marker annotation carrying constraint metadata, (2) a ConstraintValidator implementation containing the isValid() logic. The annotation must have message, groups and payload attributes — required by the Bean Validation spec. The validator can be a Spring bean — meaning it can @Autowire repositories for database-driven validation such as checking if an email is already taken.

```
// 1. Annotation
@Documented @Constraint(validatedBy = UniqueEmailValidator.class)
@Target(ElementType.FIELD) @Retention(RetentionPolicy.RUNTIME)
public @interface UniqueEmail {
String message() default "Email already registered";
Class[] groups() default {};
Class[] payload() default {};
}
// 2. Validator (Spring bean — can @Autowire)
@Component
public class UniqueEmailValidator implements ConstraintValidator {
@Autowired private UserRepository repo;
public boolean isValid(String email, ConstraintValidatorContext ctx) {
return email == null || !repo.existsByEmail(email);
}
}
// 3. Usage in DTO
@UniqueEmail
private String email;
```

Interview Questions — Chapter 7

Q1. What is the difference between @NotNull, @NotEmpty, and @NotBlank?

Ans: @NotNull: rejects null only. An empty string "" passes. @NotEmpty: rejects null AND zero-length Strings, Collections, Maps and Arrays. A whitespace-only string " " passes. @NotBlank: String-only. Rejects null, empty string "", and whitespace-only strings. It internally calls trim() before checking length. Rule of thumb: use @NotBlank on all text fields (name, description, email). Use @NotNull on numeric, boolean, date and object-type fields. Combine as needed.

Q2. How do you validate nested objects and collections?

Ans: For nested objects: add @Valid on the nested field in the parent DTO. Without @Valid on the field, the nested object's own annotations are silently ignored — a very common and hard-to-catch bug. For collections: @Valid List items — validates each element. For cross-field validation (endDate must be after startDate): add a class-level custom @Constraint or use @AssertTrue on a boolean method that performs the cross-field check.

Q3. Can validation be applied at the service layer, not just the controller?

Ans: Yes. Add @Validated to the @Service class and put @Valid on method parameters. Spring creates an AOP proxy around the service that intercepts calls and validates parameters before invoking the actual method. Failure throws ConstraintViolationException (different from the controller's MethodArgumentNotValidException). Handle it in a separate @ExceptionHandler. This is useful for ensuring business rules are enforced on service-to-service calls that bypass the HTTP layer.

CHAPTER 8

Implementing REST CRUD Operations

Layered architecture, DTO pattern, @Transactional deep dive, service design

8.1 The Layered Architecture — Why Separation of Concerns Matters

Layer	Annotation	Responsibility	What It Must NOT Do
Presentation	@RestController	Parse HTTP requests, validate input (@Valid), call service, build HTTP response	Business logic, DB queries, transaction management
Service	@Service @Transactional	Business logic, orchestration, authorization, Entity-to-DTO conversion	HTTP concerns (no HttpServletRequest usage), direct DB writes without service involvement
Repository	@Repository / JpaRepository	Data access — CRUD, queries, pagination	Business logic, validation, DTO conversion
Domain	@Entity	Database table mapping, relationships	Business methods, HTTP imports, service calls

8.2 The DTO Pattern — Why It Is Not Optional

The Data Transfer Object pattern separates your API contract (what clients see) from your database model (how data is stored). This seems like extra work but is essential for three production-critical reasons:

- Security: JPA Entities often contain sensitive fields (password hash, internal IDs, audit timestamps). Returning the Entity directly exposes ALL fields to the client — a data leak. DTOs expose only what the client needs.
- API Stability: When you change your DB schema (add a column, rename a table, change a relationship), the Entity changes. If you return Entities directly, your API changes too — breaking existing clients. With DTOs, you absorb DB changes in the service layer mapping without changing the API contract.
- Performance and Safety: Entities have lazy-loaded relationships. Accessing them outside a transaction triggers LazyInitializationException, or (worse, with Open Session in View) silently fires N+1 queries. DTOs are plain objects — no session needed, no hidden queries.

8.3 @Transactional — The Theory Every Engineer Must Know

DEEP DIVE

@Transactional wraps a method in a database transaction. Spring creates an AOP proxy around your @Service class. When the method is called on the proxy (not self-calls — that is a common bug), the proxy opens a transaction (either new or joining an existing one), calls the actual method, and then commits on success or rolls back on RuntimeException. This ensures atomicity: either ALL database operations in the method succeed together or ALL are rolled back. TRANSACTION PROPAGATION TYPES: REQUIRED (default): Join existing transaction if one exists. Start new one if not. Most common choice. REQUIRES_NEW: Always start a NEW transaction. Suspend the outer one. Use case: audit logging — audit must save even if the outer transaction rolls back. NESTED: Start a nested transaction with a savepoint. If inner rolls back, outer can continue from the savepoint. SUPPORTS: Use transaction if one exists, run without one if not. NOT_SUPPORTED: Always run without transaction. NEVER: Fail if a transaction currently exists. @Transactional(readOnly=true) for GET/read-only methods: Hibernate skips dirty checking (no need to compare entity state for changes at end of transaction), can improve performance significantly in read-heavy applications. Also signals to DB driver that no write locks are needed.

8.4 The N+1 Select Problem

N+1 problem: fetching a List of Users (1 SELECT query) where each User has a List of Orders with FetchType.LAZY. When you access user.getOrders() in code, JPA fires a separate SELECT for EACH user's orders — N additional queries. With 100 users = 101 total queries. Often invisible in dev with small data, but catastrophic in production with thousands of records and thousands of requests per second.

Solutions in order of preference: (1) JOIN FETCH in JPQL — @Query("SELECT u FROM User u JOIN FETCH u.orders WHERE u.id=:id"). (2) @EntityGraph on the repository method specifying which associations to eagerly load. (3) DTO projections — @Query returning DTO constructor expressions, selecting ONLY needed columns without loading the entity at all. Always check with spring.jpa.show-sql=true and count queries per HTTP request in development.

Interview Questions — Chapter 8

Q1. What is the N+1 select problem and how do you fix it?

Ans: N+1 happens when JPA makes 1 query to fetch a list then N separate queries for each item's lazy-loaded relationship. With 100 users and lazy orders = 101 queries. Fix options: (1) JOIN FETCH in JPQL: @Query("SELECT u FROM User u JOIN FETCH u.orders"). (2) @EntityGraph on repository method specifying eager associations. (3) DTO projections with @Query selecting only needed columns without loading entities. Always verify with spring.jpa.show-sql=true — N+1 is invisible in dev (small data, fast queries) but production-killing at scale.

Q2. What is @Transactional and why is it in the SERVICE layer not the controller?

Ans: @Transactional wraps the method in a DB transaction via Spring AOP proxy. It is in the service layer because a service method may call MULTIPLE repository operations that must be atomic — all succeed or all roll back. If @Transactional were on the controller, it would open a transaction too early (including HTTP parsing time) and keep DB connections open unnecessarily. COMMON BUG: @Transactional on private methods or same-class self-calls does NOT work — Spring AOP proxy is bypassed. Always call @Transactional methods from ANOTHER bean, not self.

Q3. Why should you never expose JPA Entities directly from REST controllers?

Ans: Three reasons: (1) Security — entities contain ALL fields including sensitive ones (password, internal IDs). (2) Lazy loading — accessing lazy fields outside a transaction throws LazyInitializationException, or with OSIV (Open Session in View) triggers hidden N+1 queries. (3) Coupling — API contract tied to DB schema. Any column rename or new field breaks the API. DTOs create a stable contract between API and client while the DB schema can evolve independently.

Q4. What is optimistic locking and when do you use it?

Ans: Add @Version Long version field to your Entity. When updating, JPA includes WHERE version=X in the UPDATE statement. If another transaction already updated the row (incremented version to X+1), the WHERE clause matches 0 rows and Hibernate throws ObjectOptimisticLockingFailureException. Catch this in your service and return 409 Conflict to the client. Use optimistic locking when concurrent updates are possible but infrequent (user profile edits). Use pessimistic locking (SELECT FOR UPDATE) when conflicts are frequent (inventory deduction, seat booking).

CHAPTER 9

Pagination & Sorting

Why unbounded queries kill production — Pageable internals, Page vs Slice, keyset pagination

9.1 The Production Disaster Story

STORY

True story (repeated at many startups): A developer builds GET /users that returns ALL users. In dev: 20 users, 50ms response, everyone is happy. Three months after launch: 500,000 users in production. A partner integration calls GET /users — it takes 45 seconds, returns 400MB of JSON, consumes 2GB of heap, triggers a full GC pause, and brings down the server. The outage lasts 4 hours. Post-mortem conclusion: "We should have paginated from day one." Pagination is not a feature — it is not optional — it is survival. Every list endpoint must paginate, period.

9.2 How Spring Resolves Pageable — The Internal Mechanism

Pageable is resolved by PageableHandlerMethodArgumentResolver, registered automatically when spring-data-web is on the classpath (included with spring-boot-starter-data-jpa). When your controller method has a Pageable parameter, Spring reads three query parameters from the request: page (default 0, zero-indexed), size (default 20), and sort (format: field,direction — e.g., ?sort=name,asc&sort;=createdAt,desc for multi-column sort). These are assembled into a PageRequest object which is the concrete implementation of the Pageable interface.

```
// Controller — Pageable auto-resolved from query parameters
@GetMapping
public Page list(
@PageableDefault(size=20, sort="createdAt", direction=Sort.Direction.DESC)
Pageable pageable) {
return userRepo.findAll(pageable).map(mapper::toDto);
}
// URL: GET /users?page=0&size;=20&sort;=name,asc&sort;=createdAt,desc
```

9.3 Page<T> vs Slice<T> — Performance Theory

This choice has significant performance implications at scale. Page<T> executes TWO SQL queries: SELECT with LIMIT/OFFSET for the data, and SELECT COUNT(*) for the total count (needed for totalPages and totalElements). The COUNT(*) query can be expensive on large tables — it must scan the entire index. Slice<T> executes ONLY ONE query — the data query with LIMIT+1 (fetches one extra row to determine if there is a next page). No COUNT(*) at all.

Feature	Page<T>	Slice<T>
SQL queries executed	2 (data + COUNT(*))	1 (data only, limit+1 trick)
Performance on large tables	Slower — COUNT is expensive	Faster — no COUNT query
Provides totalElements / totalPages	Yes	No

Provides hasNext()	Yes (calculated from count)	Yes (from fetching limit+1)
Best for	Traditional UI with page numbers	Infinite scroll, "Load More" buttons

9.4 The OFFSET Performance Problem — Why Deep Pages Are Slow

OFFSET-based pagination (Spring default) has a mathematical performance problem. SELECT * FROM users ORDER BY id LIMIT 20 OFFSET 10000 does NOT skip 10,000 rows at the index level. The database engine reads 10,020 rows, discards the first 10,000, and returns the last 20. As the page number increases, the query gets progressively SLOWER. Page 1 is fast; page 1000 is very slow.

Solution for large datasets: keyset pagination (cursor-based). Instead of OFFSET, use WHERE id > lastSeenId ORDER BY id LIMIT 20. This uses the index efficiently to start reading from the right position — O(1) regardless of cursor position. The tradeoff: no random page access (you can only go forward/backward sequentially). Twitter, Instagram and most social feeds use keyset pagination for this reason.

Interview Questions — Chapter 9

Q1. What SQL does Spring Data generate for Pageable and why does OFFSET get slower at higher page numbers?

Ans: For page=2, size=20: SELECT * FROM users ORDER BY created_at DESC LIMIT 20 OFFSET 40. And SELECT COUNT(*) FROM users. OFFSET problem: the DB reads rows sequentially from the start of the result set and discards the first 40. At page 500 (OFFSET 10000) — the DB reads 10,020 rows and discards 10,000. The larger the offset, the more work done and discarded. Solution for high-page access: keyset (cursor) pagination — WHERE created_at < :lastSeen ORDER BY created_at DESC LIMIT 20 — starts reading from the right index position directly.

Q2. How do you prevent clients from requesting extremely large page sizes?

Ans: Set spring.data.web.pageable.max-page-size=100 in properties to globally cap the size parameter. Add @PageableDefault(max=100) per controller for explicit control. Validate in service layer: if(pageable.getPageSize() > 100) throw BadRequestException. Document the max size in OpenAPI spec. Set spring.data.web.pageable.default-page-size=20 so omitting the size parameter gives a sensible default rather than Spring's internal default.

Q3. What is the difference between Page and Slice in Spring Data?

Ans: Page runs 2 queries (data + COUNT) and gives totalElements, totalPages, page number, isFirst(), isLast(). Slice runs 1 query (data only, fetches size+1 to detect next page) and gives hasNext(), hasPrevious(), content, but NOT totalElements or totalPages. Use Page for traditional paginated UIs with page numbers. Use Slice for infinite scroll or "Load More" patterns where total count is not needed — saves significant database load at scale.

CHAPTER 10

API Documentation — Swagger / OpenAPI

OpenAPI 3.0 theory, SpringDoc internals, production documentation strategy

10.1 The OpenAPI Specification — What It Is and Why It Matters

OpenAPI Specification (OAS) is a language-agnostic standard for describing HTTP APIs. It originated as the Swagger specification from SmartBear, donated to the OpenAPI Initiative (Linux Foundation) in 2015 and renamed OpenAPI. Version 3.0 (2017) is the current industry standard. An OpenAPI document is a JSON or YAML file that fully describes: all endpoints with their HTTP methods, request parameters (path, query, header, cookie), request and response body schemas with types, authentication schemes, and example values. This document is machine-readable — tools can generate client SDKs in 20+ languages, server stubs, interactive documentation (Swagger UI), test suites and API mocking from it automatically.

10.2 SpringDoc — How It Generates Documentation

DEEP DIVE

SPRINGDOC INTERNAL MECHANISM: 1. SpringDoc scans all @RestController beans in the application context at startup. 2. It reads every @RequestMapping annotation (URL, HTTP method, consumes, produces), parameter types (@PathVariable, @RequestParam, @RequestBody) and return types via Java reflection. 3. It inspects Bean Validation annotations (@NotNull, @Size, etc.) on DTO classes to add validation constraints to the schema. 4. It builds an OpenAPI 3.0 object model in memory. 5. The /v3/api-docs endpoint serializes this model to JSON on demand. 6. Swagger UI (served as static resources embedded in the springdoc JAR) loads this JSON and renders the interactive documentation. 7. Your @Operation, @ApiResponse and @Tag annotations enrich the auto-generated spec with human descriptions.

```
// Dependency: springdoc-openapi-starter-webmvc-ui:2.3.0
// Auto-exposes: /swagger-ui.html and /v3/api-docs
@Tag(name="Users", description="User management") // Groups endpoints in UI
@RestController @RequestMapping("/api/v1/users")
public class UserController {
@Operation(summary="Get user by ID", description="Returns 404 if not found")
@ApiResponse(responseCode="200", description="User found")
@ApiResponse(responseCode="404", description="User not found")
@GetMapping("/{id}")
public ResponseEntity get(@PathVariable Long id) { ... }
}
```

10.3 Production Documentation Strategy

Swagger UI should be DISABLED in production environments. Reasons: it exposes your complete API surface area to anyone who discovers the URL, reveals endpoint patterns attackers can probe, and consumes server resources. Correct approach: disable in production profile with springdoc.swagger-ui.enabled=false in application-prod.properties. Keep enabled in dev and staging. For external API documentation, export the OAS

spec as a static JSON file and host it on a dedicated documentation platform (Redocly, Postman, Stoplight). This gives you version-controlled, reviewable API documentation separated from your runtime.

Interview Questions — Chapter 10

Q1. What is the difference between Swagger and OpenAPI?

Ans: Swagger was a project by SmartBear creating a specification for describing REST APIs plus tooling (Swagger UI, Swagger Codegen). In 2015 SmartBear donated the Swagger specification to the OpenAPI Initiative (Linux Foundation). It was renamed OpenAPI Specification (OAS). Swagger 2.0 became OpenAPI 2.0. OpenAPI 3.0 (2017) added major improvements: native request body support, multiple servers, better component reuse, callbacks and links. Today: "Swagger" refers to the tooling (Swagger UI, Swagger Editor, Swagger Codegen). "OpenAPI" is the specification. In Spring Boot 3.x, use springdoc-openapi 2.x which generates OAS 3.0.

Q2. How do you secure Swagger UI with JWT in Spring?

Ans: In your OpenApiConfig @Bean: add .addSecurityItem(new SecurityRequirement().addList("bearerAuth")) to the OpenAPI object and in .components() add a SecurityScheme with type HTTP, scheme "bearer" and bearerFormat "JWT". This adds an Authorize button in Swagger UI where users paste their JWT token. Annotate secured endpoints with @SecurityRequirement(name="bearerAuth"). For dev workflow: add a /auth/login endpoint visible in Swagger UI, call it to get a token, paste it in Authorize — then all secured endpoint tests include the Bearer token automatically.

CHAPTER 11

Spring REST Best Practices

URI design, versioning philosophy, security checklist, idempotency, rate limiting — thinking like a senior engineer

11.1 URI Design — The Philosophy

REST URI design is deceptively simple but frequently done wrong. The guiding principle: URIs identify RESOURCES (nouns), not ACTIONS (verbs). The HTTP method expresses the action. When you feel the urge to put a verb in your URI (/getUsers, /deleteOrder, /fetchProducts), stop and ask: what resource am I operating on? Then use the HTTP verb.

Rule	Wrong (Anti-Pattern)	Correct (REST)	Reason
Nouns not verbs	GET /getUsers	GET /users	Verb is redundant — GET already means retrieve
Plural nouns	GET /user/42	GET /users/42	Resources are collections; /user is ambiguous
Lowercase + hyphens	GET /UserProfile	GET /user-profiles	URIs are case-sensitive; hyphens aid readability
Hierarchy for relations	GET /getUserOrders	GET /users/42/orders	Nesting shows the relationship clearly
Query params for filtering	GET /users/active	GET /users?status=active	"active" is a filter, not a sub-resource
No trailing slash	GET /users/	GET /users	Trailing slash implies a subdirectory
Version in path	GET /users	GET /api/v1/users	Enables backward-compatible evolution

11.2 API Versioning — Four Strategies Compared

API versioning enables you to evolve your API without breaking existing clients. It is not optional — every production API will eventually need breaking changes. The question is not WHETHER to version, but HOW.

Strategy	Format	Pros	Cons
URI Path	GET /api/v1/users	Simple, visible, cacheable, browser-testable, works with all tools	URI is "impure" per strict REST theory
Request Header	X-API-Version: 1	Clean URIs, REST-pure	Hard to test in browser, hidden from logs and documentation
Query Parameter	GET /users?version=1	Easy to test in browser	Messy URLs, cache key complications
Content-Type	Accept: app/vnd.co.v1+json	Most REST-pure, true content negotiation	Very complex, almost never implemented correctly in practice

TIP	INDUSTRY RECOMMENDATION: URI versioning (/api/v1/) is the most widely used strategy. It is explicit, cacheable, works with every tool (browsers, Postman, curl, API gateways, CDNs), and is used by Netflix, Stripe, Twilio, Google and AWS. The "impure URI" argument is academic — pragmatism wins in production.

11.3 Idempotency — Making POST Safe to Retry

Network failures are inevitable. When a client sends POST /orders and the network dies before receiving the response, the client does not know if the order was created. If it retries without any idempotency mechanism, it creates a duplicate order. This is a serious production bug in e-commerce and payment systems. Solution: Idempotency keys. The client generates a UUID and sends it in the Idempotency-Key header. The server checks Redis or a DB table if this key was seen before. If yes, return the cached response without reprocessing. If no, process normally, cache the response with the key (for 24 hours), and return it. Stripe, PayPal, Square and most payment APIs implement this as a standard feature.

11.4 Rate Limiting — Protecting Your API

Rate limiting prevents API abuse, DoS attacks and ensures fair resource allocation across clients. Approaches: (1) Bucket4j library — token bucket algorithm, integrates with Spring via a filter or interceptor. Configurable per IP, per API key, per user. (2) Spring Cloud Gateway — built-in rate limiting with Redis Lua scripts for atomic token counting. (3) Infrastructure-level — AWS API Gateway, Kong or NGINX rate limiting — handle rate limiting outside your application entirely. When a request is rate-limited, return 429 Too Many Requests with a Retry-After header telling the client exactly when to retry. Never return 400 — clients need to know to back off, not that their request was malformed.

11.5 Production Readiness Checklist

- **checkmark** DTOs used everywhere — JPA Entities NEVER exposed directly from controllers
- **checkmark** Global exception handler with @RestControllerAdvice — zero scattered try-catch
- **checkmark** All endpoints return semantically correct HTTP status codes
- **checkmark** @Valid on all @RequestBody and @RequestParam parameters that require validation
- **checkmark** Pagination on ALL list endpoints — no unbounded queries ever
- **checkmark** API versioning in URI (/api/v1/) — enables future evolution without breaking clients
- **checkmark** OpenAPI/Swagger documentation — disabled in production, enabled in dev/staging
- **checkmark** Input sanitization — @Pattern or sanitizer library prevents XSS and SQL injection
- **checkmark** No stack traces returned to client — only generic message, full log server-side
- **checkmark** Authentication (JWT) + Authorization (@PreAuthorize) on all secured endpoints
- **checkmark** Rate limiting — per-IP and per-user, returns 429 with Retry-After header
- **checkmark** Correlation ID header — unique request ID for distributed tracing across microservices
- **checkmark** Health endpoint via Spring Actuator (/actuator/health) for load balancer checks
- **checkmark** Idempotency keys supported on POST endpoints that create critical resources
- **checkmark** spring.jpa.hibernate.ddl-auto=validate in production — never create or drop
- **checkmark** Secrets in environment variables or secrets manager — never hardcoded in code
- **checkmark** Swagger UI disabled in production — springdoc.swagger-ui.enabled=false

checkmark CORS restricted to specific origins — never allowedOrigins("*") with credentials

Interview Questions — Chapter 11

Q1. How do you handle backward-compatible API changes vs breaking changes?

Ans: Backward-compatible (non-breaking) changes can be made to existing API version: adding new optional response fields, adding new optional query parameters, adding entirely new endpoints. Breaking changes REQUIRE a new version: removing or renaming response fields, changing field data types, removing endpoints, changing the HTTP method for an existing endpoint. Strategy: (1) Deploy v2 alongside v1 simultaneously. (2) Mark v1 as deprecated with Deprecation response header and Sunset date header. (3) Give clients minimum 3-6 months migration window. (4) Monitor v1 traffic via metrics. (5) Retire v1 only when traffic is near zero.

Q2. What is CORS and how do you configure it properly in Spring?

Ans: CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks JavaScript on domain A from calling APIs on domain B without explicit server permission. The browser sends a preflight OPTIONS request first to check if the cross-origin call is allowed. In Spring: @CrossOrigin(origins="https://yourapp.com") on specific controllers, or global config via WebMvcConfigurer.addCorsMappings(). Production security: allowedOrigins must list specific domains, never "*" with allowCredentials(true) — that is a security vulnerability. maxAge(3600) caches the preflight response for 1 hour to avoid preflight overhead on every request.

Q3. How do you design an API for a long-running operation like report generation?

Ans: Use the async/polling pattern: POST /reports → returns 202 Accepted immediately with a Location header pointing to /reports/jobs/abc123. Start the long-running work asynchronously (Spring @Async or a message queue like Kafka/RabbitMQ). Client polls GET /reports/jobs/abc123 which returns status: PENDING, PROCESSING, or COMPLETED with a downloadUrl. This avoids HTTP gateway timeouts (most gateways timeout at 30-60 seconds), enables the client to continue other work, and allows you to scale report generation independently on dedicated worker servers.

Q4. What is the role of Spring Actuator in production?

Ans: Spring Actuator exposes operational endpoints for monitoring and management. Key ones: /actuator/health — load balancer and Kubernetes liveness/readiness probes (return UP/DOWN with DB, cache, and external service status). /actuator/metrics — Prometheus-compatible metrics (request count, latency percentiles, JVM memory, DB connection pool size). /actuator/info — application version, build time. /actuator/loggers — change log levels at runtime without restart — invaluable for debugging production issues. Security: expose only /health publicly. All other actuator endpoints must be protected — /actuator/env exposes all configuration including secrets if misconfigured.

Quick Reference Appendix

All Spring REST Annotations — Complete Reference

Annotation	Layer	Purpose and When to Use
@RestController	Controller	Marks REST controller class. Combines @Controller + @ResponseBody.
@RequestMapping	Controller	Maps URL pattern + HTTP method. Class-level = base path for all methods.
@GetMapping, @PostMapping, @PutMapping, @PatchMapping, @DeleteMapping	Controller	Shorthand annotations for specific HTTP methods on top of @RequestMapping.
@PathVariable	Controller	Extracts a URI template variable {name} from the path segment.
@RequestParam	Controller	Extracts query string parameter. Supports defaultValue and required=false.
@RequestBody	Controller	Deserializes HTTP request body via Jackson ObjectMapper.
@ResponseStatus	Controller	Sets the HTTP status code. Use 201 for POST, 204 for DELETE.
@Valid / @Validated	Controller / Service	Triggers Bean Validation on the annotated parameter. Required on @RequestBody.
@RestControllerAdvice	Global	Global exception handler for all controllers. Returns JSON errors.
@ExceptionHandler	Global	Maps exception type to handler method inside @ControllerAdvice.
@CrossOrigin	Controller	Configures CORS for a controller or method. Restrict to specific origins.
@PageableDefault	Controller	Sets default pagination values (page, size, sort) for Pageable parameter.
ResponseEntity	Controller	Return type for full control over HTTP status, headers, and body.
@Transactional	Service	Wraps method in DB transaction. Use readOnly=true for GET methods.
@JsonProperty / @JsonIgnore / @JsonFormat	DTO	Jackson annotations controlling JSON field name, exclusion, and format.
@NotNull / @NotBlank / @Size / @Email / @Valid	DTO	Bean Validation constraints. Use @NotBlank on text, @NotNull on objects.

TIP

FINAL THOUGHT FROM A SENIOR ENGINEER: The mark of expertise is not knowing every annotation by heart — it is understanding WHY each one exists, WHAT happens internally when you use it, and WHEN not to use it. REST is not about memorizing rules. It is about understanding architectural constraints that produce scalable, maintainable systems. Every concept in these notes — from statelessness to idempotency — exists to solve a real problem at real scale. These patterns are used by Netflix, Google, Amazon and every serious engineering team. Now you understand them deeply. Go build systems that last. ■